

Flowvelly Healthcare Prod GKE Inference Incident Response

Flowvelly Healthcare Prod GKE Inference Incident Response

Context and assumptions

- Flowvelly Healthcare GCP org has a folder hierarchy with `qa`, `preprod`, `prod` under a shared-healthcare folder.
- Production inference service runs on GKE in a host project attached to a Shared VPC.
- Workloads include GKE, Cloud SQL Postgres, and Cloud Storage.
- A partner team merged a host-project networking Terraform change this morning.
- No application deploy happened today.
- Incident began ~30 minutes ago: prod inference returns intermittent 5xx at ~1 in 6 requests, while successful requests show normal latency.
- Cloud SQL CPU is normal.
- PHI-bordering data is in scope, so verbose request/response payload logging is not permitted.

Prioritized triage

1. Confirm impact and scope first

- Verify whether the failure rate is stable or increasing and the exact scope: prod only, specific service, or some regions/zones.
- Capture incident time window and correlate with the partner networking change timestamp.
- Inform incident commander and document the initial hypothesis: network/host-level change affecting a subset of backends.

2. Check the load balancer and backend health

- Inspect the HTTPS/HTTP(S) LB backend service and NEG endpoint health.
- Query `gcloud compute backend-services get-health` for the prod backend service.
- Look for `UNHEALTHY` or `DRAINING` endpoints in one or more zones.
- Collect LB metrics: 5xx count, 5xx ratio, backend connection failures, backend latency, unhealthy host count.

3. Evaluate GKE readiness/probe state

- Check `kubectl get pods -n <prod-namespace>` for the inference deployment.
- Inspect `kubectl describe pod` and `kubectl get events --all-namespaces` for readiness/liveness failures, OOMKills, or node affinities.
- Confirm whether the traffic distribution is causing only some pods to fail.

4. Assess Cloud SQL connectivity and connection limits

- Check Cloud SQL `database\_connections`, `active\_connections`, `failed\_connections`, and `connection\_count` metrics.
- Inspect `pg\_stat\_activity` for blocked connections or repeated reconnects.
- Verify whether a small fraction of requests is failing due to connection exhaustion or proxy failures rather than CPU.

5. Review host project networking changes and Shared VPC path

- Examine the network Terraform change from this morning: firewall rules, routes, NAT configs, VM tags, and subnetwork IAM bindings.
- Check Cloud Audit Logs for the exact apply time and changed resource types.
- Validate Shared VPC firewall rules for health checks, ingress to node ports, Cloud SQL traffic, and egress through Cloud NAT.

6. Inspect logging/monitoring with PHI-safe scope

- Use structured audit-safe logs: status codes, service name, request IDs, error categories, subsystem, and endpoint, but not payloads.
- Query HTTP LB logs for 5xx counts by backend name/zone and by `request\_path` or `response\_code` only.
- Check Error Reporting and Monitoring alert history for sudden 5xx spikes or backend health changes.
- Avoid enabling verbose payload logging; if additional context is needed, use header-only or metadata-only logging.

What “1 in 6” implies

- A ~16.7% failure rate is too high for random transient errors and strongly suggests a systemic issue affecting a subset of endpoints or a specific path.
- Possibilities:
 - One of six backend pods or endpoint groups is failing.
 - One of six zones or nodes has a bad network path or firewall filtering.
 - One of six Cloud NAT IPs or ephemeral port ranges is dropping connections.
- It is less likely to be a global resource exhaustion issue across all pods, because normal latency is preserved for the successful 5/6 traffic.

Hypotheses

1. Host-project networking Terraform change introduced a firewall/route/NAT issue that affects only some node pools, zones, or egress paths.
2. Some NEG endpoints are unhealthy or partially draining, causing 1 in 6 requests to hit a bad subset of backends.
3. Cloud SQL connection handling is intermittent on a subset of app instances or proxy paths, without CPU saturation.
4. GKE readiness/liveness probes or node network configuration are failing for a subset of nodes.

Likely root cause and confirmation strategy

Likely root cause

- A shared-host networking change altered the data path for the GKE inference service and produced inconsistent backend health for a subset of endpoints.
- This is consistent with a partner Terraform change this morning, the absence of an app deploy, and the ~1-in-6 failure pattern.

Confirm by pulling these GCP signals

- LB backend health and NEG status for prod service.
- `kubectl describe` on failing inference pods to detect readiness/liveness retries or node-level errors.
- Cloud Monitoring 5xx ratios and backend service error metrics.
- Cloud SQL connection metrics: failed connections, connection exhaustion, connection resets.
- VPC Flow Logs / network connectivity tests only if they can be used safely without payload data.
- Terraform change history / Cloud Audit Logs for the host-project networking change.

Immediate mitigation vs durable fix

Immediate mitigation

- If a particular backend zone/pod set is unhealthy, remove or drain that subset from the LB and scale up healthy pods.
- If the host networking change is confirmed, rollback the faulty firewall/route/NAT change immediately and ensure Shared VPC connectivity is restored.
- If Cloud SQL connection exhaustion is suspected as a secondary contributor, increase connection pool limits or reduce restart churn on app pods.
- Avoid broad payload logging. Instead, add a temporary redacted error-only log path or increase sampling of existing structured logs.

Durable fix

- Lock down host project networking Terraform changes with an explicit review workflow and Shared VPC governance.
- Add a prod runbook for intermittent 5xx on the inference service that starts with LB/NEG health and host-project network change correlation.
- Add alerts for:
 - HTTP(S) LB 5xx rate > 10% for 5 minutes.
 - NEG endpoint health mismatch or unhealthy count > 0.
 - Cloud SQL failed connection attempts or connection count > 80% of max.
 - GKE readiness probe failures across more than one pod.
- Harden PHI-safe logging by collecting only headers, status, request IDs, and error categories.

Coordination plan

- Notify the partner host-project networking team immediately, share the exact change window, and ask them to pause additional network changes.
- In parallel, ask the app team to confirm there were no production code pushes and to provide any recent pod-level or readiness probe changes.
- Use the host/app team separation: partner team owns Shared VPC / firewall / NAT; app team owns GKE deployment, probes, and Cloud SQL schema.
- Keep all discussion in an auditable channel and preserve the change timeline for compliance evidence.
- If the downstream fix requires a rollback, coordinate an emergency rollback with explicit approval and documentation for audit.

Runbook and alert additions

Runbook steps for this incident type

1. Check production LB 5xx rate and backends health.
2. Check GKE pod readiness/liveness events for inference deployment.
3. Check Cloud SQL connection metrics and ``pg_stat_activity``.
4. If the issue began after a host-project network change, inspect the exact Terraform diff and revert if needed.
5. Confirm PHI-safe logs are active; do not enable payload capture.
6. Escalate to partner network and app teams with the suspected failure domain.
7. Document the timeline, decisions, and rollback actions for audit.

Alert definitions

- Prod HTTP(S) LB 5xx rate > 15% sustained for 3 minutes.
- PROD NEG unhealthy endpoint count > 0.
- Cloud SQL failed connection count spike > 5 over 5 minutes.
- GKE deployment readiness probe failure rate > 10% for any inference deployment.

AI assistant iteration, answer, and critique

Scenario pasted into an assistant

```text

14:20 US-hours: a Flowvelly Healthcare GKE inference service in prod returns intermittent 5xx — ~1 in 6 requests; good requests have normal latency; began ~30 min ago. No deploy today, but a partner merged a host-project networking Terraform change this morning. Cloud SQL CPU is fine. The app team says "nothing changed our side." PHI-bordering, so you can't enable verbose payload logging. Walk us through your prioritized triage — what you check first and why, hypotheses, the GCP signals you'd pull (LB/backend & NEG endpoint health, GKE events/readiness, Cloud SQL connection limits, VPC/firewall/Cloud NAT, Logging/Monitoring, Error

Reporting), what "1 in 6" specifically implies, likely root cause + how to confirm, immediate mitigation vs. durable fix, and how you coordinate the partner and app teams and add a runbook/alert.

...

### ***Example AI assistant answer***

- Check Cloud Logging, Monitoring, Error Reporting, and Kubernetes event logs.
- Inspect Cloud SQL metrics and see if database connections or CPU are spiking.
- Review GKE pod status and deployment rollout.
- Validate firewall rules and VPC connectivity.
- Likely cause: either application bug, Cloud SQL connection issue, or load balancer misconfiguration.
- Mitigation: restart pods, adjust database connections, and consider increasing logging.

### ***Where this answer is shallow or unsafe for Flowvelly Healthcare***

- It is an unprioritized checklist. It does not emphasize the host-project networking change or the likelihood that 1 in 6 errors points to a subset of backends, not a generic app failure.
- It misses that the PHI-bordering environment forbids verbose payload logging and should use redacted structured metadata instead.
- It invents a generic triage path without calling out LB/NEG health, Shared VPC firewall/NAT path validation, or partner■host coordination.
- It treats the issue as a generic app bug and does not surface the critical cross-team handoff between partner network and app teams.
- It does not define durable alerts or runbook steps appropriate for healthcare audit evidence.

## **AI Companion Note**

- I used AI as a drafting assistant and then critically reviewed its output against the healthcare incident context.
- I corrected two concrete shortfalls: the missing emphasis on the 1-in-6 failure signal and the PHI-safe logging restriction.
- I added architecture-level context around Shared VPC / host-project networking Terraform governance and the need for auditable coordination with partner teams.
- I also added specific runbook alerts, a prioritized incident triage order, and a confirmation strategy tied to LB backend/NEG health and Cloud SQL connection metrics.
- This note is intentionally honest: the AI provided a broad checklist, but I turned it into a targeted, risk-aware response for a healthcare-sensitive GCP estate.